

热身赛(01)T2 《序列分段》题解

一、题意（简要）

给你一串不降的数列 $a_1 \leq a_2 \leq \dots \leq a_n$ 。对每个段数 $m = 1, 2, \dots, k$ ，把它切成 m 段（段可以空），第 i 段里的每个数都加上 i 。加完后第 j 个数变成 $a_j + c_j$ （ c_j 就是它所在段的编号），问 $\sum_j (a_j + c_j)^2$ 最大是多少，记为 q_m 。每次询问互不干扰，最后只要输出

$$\left(\sum_{m=1}^k q_m \right) \bmod 998244353$$

说白了：把“每种段数下的最大平方和”全加起来，再对 **998244353** 取个模。

二、数据范围与突破口

测试点	分数	$n \leq$	$k \leq$	$ a_i \leq$	特殊性质
1 ~ 3	15	12	12	1000	无
4 ~ 6	15	1000	1000	1000	无
7 ~ 8	10	10^6	10^6	10^7	$a_i \geq 0$
9 ~ 12	20	10^6	1000	10^7	无
13 ~ 20	40	10^6	10^7	10^7	无

盯住 $k \leq 10^7$ ：任何“老老实实枚举每个 m ”的做法都会超时。所以必须把对 m 的求和一次性算掉——这就是本题的命门，也是最后一步优化的方向。

三、预备知识：模意义下怎么“除以 6”

很快要用的平方和公式 $S(x) = \frac{x(x+1)(2x+1)}{6}$ 里有个“除以 6”。但答案要对 $p = 998244353$ 取模，而模意义下没有除法：既不能先算完再除（真实值会爆 `long long`），也不能拿取模后的数直接除以 6。正确做法是用乘法逆元把除法变成乘法。

乘法逆元

若在模 p 下 $a \cdot x \equiv 1 \pmod{p}$ ，就把 x 记作 a^{-1} 。于是“除以 a ”等价于“乘 a^{-1} ”：

$$\frac{b}{a} \equiv b \cdot a^{-1} \pmod{p}$$

逆元怎么求？用费马小定理：当 p 是质数、且 a 不是 p 的倍数时， $a^{p-1} \equiv 1 \pmod{p}$ ；两边同乘 a^{-1} ，立刻得到 $a^{p-2} \equiv a^{-1} \pmod{p}$ 。所以只要做一次快速幂算出 $a^{p-2} \bmod p$ ，就拿到了逆元，代价仅 $O(\log p)$ 。

本题 $p = 998244353$ 是质数、6 不是它的倍数，逆元确实存在： $6^{-1} \equiv 6^{p-2} \equiv 166374059 \pmod{p}$ 。代码里咱们用快速幂现场求，不把这个数写死——更通用也不易抄错；想验算就检查 $6 \times 166374059 \equiv 1 \pmod{p}$ 。

快速幂的原理：把指数按二进制拆开，边平方边乘。例如 $a^{13} = a^8 \cdot a^4 \cdot a^1$ ，只取二进制为 1 的那些位。

四、第一层：所有数都非负（ $a_i \geq 0$ ）

先别急着想满分，咱们把负数扔掉，只看 $a_i \geq 0$ 。

固定一个数 $a \geq 0$ ，把它放进第 c 段后变成 $a + c$ 。因为 $a \geq 0, c \geq 1$ ，所以 $a + c$ 始终为正，并且随 c 增大而增大，平方当然也跟着增大：

$$a + m \geq a + 1 > 0 \implies (a + m)^2 \geq (a + 1)^2$$

所以：每个非负数都该拿**能拿到的最大加数**，也就是全部塞进最后一段（加 m ），前面各段留空，合法性毫无问题。

结论 1（非负情形）

$q_m = \sum_j (a_j + m)^2$ ，并且每个数 a 在所有 m 上的总贡献为

$$\sum_{m=1}^k (a + m)^2 = \sum_{t=a+1}^{a+k} t^2 = S(a + k) - S(a)$$

其中 $S(x) = \sum_{i=1}^x i^2$ 是平方和前缀：

$$S(x) = \frac{x(x+1)(2x+1)}{6} \quad (x \geq 0, S(0) = 0)$$

这里用了“交换求和顺序”：本来是先固定 m 、对所有数求和（得 q_m ），现在改成先固定一个数 a 、对 m 求和。这一换， $\sum_m (a + m)^2$ 就变成一段连续整数的平方和，能用 S 的闭式一步算出——这是把 $O(nk)$ 压成 $O(n)$ 的关键技巧。

小 验 算： $[1, 2, 2], k = 3$ 。 $a = 1$ ： $S(4) - S(1) = 30 - 1 = 29$ ； 每个 $a = 2$ ： $S(5) - S(2) = 55 - 5 = 50$ 。合计 $29 + 50 + 50 = 129$ 。

五、第二层：所有数都为负（ $a_i < 0$ ）

现在反过来，假设全是负数。麻烦在哪？

$(a + c)^2$ 不再单调，而是个“V 形”。拿 $a = -3, m = 5$ 感受一下： c 取 $1 \sim 5$ 时， $a + c = -2, -1, 0, 1, 2$ ，平方依次是 $4, 1, 0, 1, 4$ 。两头大、中间小——因为抛物线开口向上（凸函数），它在一段区间上的最大值一定在端点取到。所以每个数要么放进第 1 段（加 1），要么放进第 m 段（加 m ），中间的段永远别放。

那什么时候选 1、什么时候选 m ？直接比大小（记 $a = -t, t > 0$ ）：

$$\begin{aligned} (a + 1)^2 &\geq (a + m)^2 \\ \iff (a + 1)^2 - (a + m)^2 &\geq 0 \\ \iff (1 - m)(2a + 1 + m) &\geq 0 \\ \iff 2a + 1 + m &\leq 0 \\ \iff a &\leq -\frac{m + 1}{2} \end{aligned}$$

第三个箭头用了 $m \geq 1$ 故 $1 - m \leq 0$ （两边同除以负数要变号）。把 $a = -t$ 代入，条件就是 $-t \leq -\frac{m+1}{2}$ ，即

$$\text{选 } c = 1 \iff m \leq 2t - 1; \quad \text{选 } c = m \iff m \geq 2t$$

边界 $m = 2t - 1$ 时两者相等，归到哪边都一样。

关键一步：这样分配一定合法吗？

不同数的 t 不一样，想加的数也可能不同——能同时实现吗？**能，而且是自动的。** 因为数列不降：越靠前的数越负（ t 越大），越容易满足 $m \leq 2t - 1$ ，于是它们该加 **1**；越靠后的数越接近 **0**，该加 m 。所以只要把前面的数全放进第 **1** 段、后面的数全放进第 m 段、中间各段全部留空，就是一个货真价实的合法分段：

$$\underbrace{[a_1, \dots, a_p]}_{\text{第 1 段, 加 1}} \quad [] \cdots [] \quad \underbrace{[a_{p+1}, \dots, a_n]}_{\text{第 } m \text{ 段, 加 } m}$$

每个数都拿到了自己想要的加数，**互不冲突**——这是本题最漂亮的“啊哈”点。

于是对单个负数 $a = -t$ ，把它在所有 $m = 1 \sim k$ 上的贡献加起来，分两种情形（这正是代码里的两个分支）：

- 若 $k \leq 2t - 1$ ：从头到尾都选 $c = 1$ ，贡献 $(a + 1)^2 \cdot k = (t - 1)^2 k$ 。
- 若 $k \geq 2t$ ：前 $2t - 1$ 个 m 选 **1**，贡献 $(t - 1)^2(2t - 1)$ ；之后 $m = 2t \sim k$ 选 m ，此时 $a + m = m - t$ ，贡献 $\sum_{m=2t}^k (m - t)^2 = S(k - t) - S(t - 1)$ 。

六、第三层：统一起来（混合情形）

把前两层合并，其实只得到一条**统一的规则**——不管正负，每个数都只在**第 1 段**和**第 m 段**之间二选一：

$$q_m = \sum_j \max\left((a_j + 1)^2, (a_j + m)^2\right)$$

- $a \geq 0$ ：恒有 $(a + m)^2 \geq (a + 1)^2$ ，永远进第 m 段（属于后缀）；
- $a < 0$ ：按 $m \leq 2t - 1$ 还是 $m \geq 2t$ 决定进第 **1** 段还是第 m 段（属于前缀）。

因为数列不降，负数自动堆成前缀、非负数自动堆成后缀，仍是“前缀放第 **1** 段、后缀放第 m 段、中间留空”的合法分段。所以混合情形不需要任何新东西，只是把两套公式拼起来。

最终公式（每个数 a 的总贡献 $g(a)$ ）

$$g(a) = \begin{cases} S(a + k) - S(a), & a \geq 0, \\ (t - 1)^2 k, & a = -t < 0, k \leq 2t - 1, \\ (t - 1)^2(2t - 1) + S(k - t) - S(t - 1), & a = -t < 0, k \geq 2t, \end{cases}$$

答案就是 $(\sum_j g(a_j)) \bmod 998244353$ 。复杂度 $O(n)$ 。

七、用样例完整走一遍

样例 $[-3, 1, 2, 2]$ ， $k = 3$ 。把它按“元素 $\times m$ ”摊开（格子里的数是该元素在该 m 下取到最优加数后的平方）：

元素 \ m	$m = 1$	$m = 2$	$m = 3$	行和 = 该元素贡献
$a = -3$ ($t = 3$, $k \leq 5$ 恒取 $c = 1$)	$(-2)^2 = 4$	4	4	12
$a = 1$ (非负, 取 $c = m$)	$2^2 = 4$	$3^2 = 9$	$4^2 = 16$	29
$a = 2$ (非负, 取 $c = m$)	$3^2 = 9$	$4^2 = 16$	$5^2 = 25$	50
$a = 2$	9	16	25	50
列和 = q_m	$q_1 = 26$	$q_2 = 45$	$q_3 = 70$	141

这张表就是整道题的骨架：每一列的和就是题目定义的 q_m (**26, 45, 70**，与题面一致)；每一行的和就是咱们算法算的“单个元素的贡献”。所谓**交换求和顺序**，说白了就是“不按列求和，改按行求和”——列要枚举 m

(会超时)，行却能用 S 的闭式一步算完。

八、完整代码与逐块讲解

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 998244353;

// 快速幂：求  $a^e \bmod MOD$ 
ll mod_pow(ll a, ll e) {
    ll res = 1;
    while (e) {
        if (e & 1) res = res * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return res;
}

// 费马小定理：MOD 为质数时， $a^{(MOD-2)}$  就是  $a$  的乘法逆元
ll inv(ll a) {
    return mod_pow(a, MOD - 2);
}

ll INV6; // 6 的逆元，main 里求一次即可

// 平方和前缀  $S(x) = 1^2 + 2^2 + \dots + x^2 \quad (x \geq 0)$ 
ll sum_sq(ll x) {
    if (x < 0) return 0;
    x %= MOD;
    return x * (x + 1) % MOD * (2 * x + 1) % MOD *
        INV6 % MOD;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    INV6 = inv(6); // 求一次，之后反复使用
    ll n, k; cin >> n >> k;
    ll ans = 0;
    for (ll i = 0; i < n; i++) {
        ll x; cin >> x;
        if (x >= 0) {
            // 非负：全放最后一段，加 m
            ans = (ans + sum_sq(x + k) - sum_sq(x) + MOD) % MOD;
        } else {
            ll t = -x; //  $t = |a| > 0$ 
            if (x + k < t) { //  $k \leq 2t-1$ ：全取  $c=1$ 
                ll v = x + 1;
                ans = (ans + v * v % MOD * k) % MOD;
            } else { //  $k \geq 2t$ ：前  $2t-1$  个取 1，其后取 m
                ll small = (t - 1) * (t - 1) % MOD * (2 * t - 1) % MOD;
```

```

        ll big = (sum_sq(x + k) - sum_sq(t - 1) + MOD) % MOD;
        ans = (ans + small + big) % MOD;
    }
}
cout << ans << "\n";
return 0;
}

```

1. **mod_pow** 与 **inv**：快速幂求 $a^e \bmod p$ （二进制拆分，边平方边乘， $O(\log p)$ ）；**inv(a)** 用费马小定理返回 a^{p-2} ，即 a 的乘法逆元。**INV6** 只在 **main** 开头求一次，不要写死成魔法数。
2. **sum_sq(x)**：返回 $S(x) \bmod p$ 。两个要点：① 先 $x \% MOD$ 再代入—— S 是多项式、 6 在模 p 下可逆，先约简自变量结果不变；② $S(x)$ 真实值在 $x \sim 2 \times 10^7$ 时约 10^{21} ，早就爆 **long long**，必须边乘边取模。 $x < 0$ 返回 0 兜底。
3. 主循环：边读边算，不用存数组，空间 $O(1)$ 。每个数独立算贡献累加进 **ans**。
4. 非负分支： $\text{sum_sq}(x+k) - \text{sum_sq}(x)$ 就是结论 1；**+MOD** 再 **%MOD** 防止相减出负数。
5. 负数分支的判定： $x + k < t$ 即 $-t + k < t$ ，也就是 $k < 2t$ ，整数意义下等价于 $k \leq 2t - 1$ ——正好“全取 $c = 1$ ”，贡献 $(a + 1)^2 k$ ($v=x+1$)。
6. 否则 ($k \geq 2t$)：**small** 是 $(t - 1)^2(2t - 1)$ ；**big** 是 $S(k - t) - S(t - 1)$ （注意 $x+k=k-t$ ）。两处乘法都先取模再乘，防溢出。

九、复杂度与易错点

- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。
- 比大小必须用真实整数： $x + k < t$ 这类判定绝不能在取模后做——模运算不保序。
- 溢出： $S(x)$ 的真实值会爆 **long long**，必须模意义下用逆元 **INV6** 算； $(t-1)^2(2t-1)$ 中 t 可达 10^7 ，也要先 **%MOD** 再乘。
- 逆元的前提：费马小定理要求 p 是质数且 a 不是 p 的倍数（本题 $p = 998244353$ 满足）。若模数不是质数，要改用扩展欧几里得算法求逆元。
- 边界： $a = -1$ ($t = 1$) 时 $(t - 1)^2 = 0$ 自然正确； $k = 2t - 1$ 时两分支结果相同，无需特判。
- 校验：样例输出 **141**，与题面 $q_1 + q_2 + q_3 = 26 + 45 + 70$ 一致。

十、拿不到正解怎么办：部分分路线

比赛时一时想不出也别交白卷，按数据范围从低到高拿分就行。

分层拿分节奏

- 第 1~3 点 ($n, k \leq 12$, 15 分)：暴搜 / DP 枚举所有合法分段，算出每个 q_m 。重点不是算法，而是跑出自答案后“看”规律：每个数都只进第 1 段或第 m 段。
- 第 4~6 点 ($n, k \leq 1000$, 累计 30 分)：套统一公式 $q_m = \sum_j \max((a_j + 1)^2, (a_j + m)^2)$, $O(nk) \leq 10^6$ 。
- 第 7~8 点 ($a_i \geq 0$, 再 +10 分)：用非负闭式 $S(a + k) - S(a)$, $O(n)$ 。
- 满分 (13~20 点)：把对 m 的求和也交换顺序化成闭式，做到 $O(n)$ 。

口诀：先拿能拿的分 → 观察规律 → 公式化 → 消去 m 的枚举。正解往往藏在为拿部分分写的暴搜里。

十一、复盘：可以带走的思维工具

最后把全文用到的“套路”收拢一下。下次遇到新题，不妨按这张表逐个检查：

思维工具	在本题哪里用	如果没有它
分层推进：先只看全非负、再只看全负、最后把两套公式合并	第四、五、六节	一上来就面对正负混杂，容易被吓住，也很难发现“两端最优”
先贪心，再验证合法性：先让每个数自由取最优，再看能否拼成合法分段	第五节：每个数各自取最优加数后，前缀放第 1 段、后缀放第 m 段、中间留空，正好合法	得到一堆“切不出来”的理论最优，答案直接错
凸函数端点性： $(a + c)^2$ 开口向上，闭区间最值在端点	第四、五节：每个数只可能进第 1 段或第 m 段	误以为要在 1 ~ m 中间找最优，白白增加讨论量
交换求和顺序： $\sum_m \sum_j \rightarrow \sum_j \sum_m$ ，把“对 m ”变成一段连续平方和	第三、四节的闭式；第六节表格里“按行求和”的直观解释	只能 $O(nk)$ 枚举每个 m ， k ≤ 10 ⁷ 必然超时

另外两件赛场小事：一是**手算验算**——像第七节那样列个表，样例过一遍再写代码；二是**对拍**——用第一档 **n, k ≤ 12** 的暴力当对拍器，正解写完跑几组随机数据，稳稳当当再提交。